HCMC University of Technology, VNU

Software Architecture

Lecturer: Thai-Minh Truong, PhD.

Email: thaiminh@hcmut.edu.vn

Aims

- The goal of this course is to provide master's students with fundamentals and principles of software design with an emphasis on software architectural design.

Outline

- The course revisits cross-paradigm principles of software design such as high cohesion & low coupling, modularity and separation of concerns followed by advanced topics in software design including the SOLID principles.
- Tatics & views in software architectural design.

Learning outcomes

No.	Course learning outcomes (CLO)	CLO assessment
L.O.1	Understand the need for having a good design in software engineering	Final exam
L.O.2	Effectively apply best-in-class software architecture design tactics to modern software-intensive systems	Assignment
L.O.3	Represent multiple views for an architectural design	Assignment
L.O.4	Follow design principles (e.g., SOLID) in making detailed design	Final exam

Textbook/reference book

- [1] Robert C. Martin, Clean Architecture: A Craftsman's Guide to Software Structure and Design, Pearson; 1st edition (September 10, 2017), ISBN 978-0134494166
- [2] Paul Clements, Felix Bachmann, Len Bass, David Garlan, James Ivers, Reed Little, Paulo Merson, Paulo Merson, Robert Nord, Judith Stafford, Documenting Software Architectures: Views and Beyond, 2nd edition, 2011, Addison-Wesley Professional, ISBN 978-0321552686
- [3] Derick Bailey, "S.O.L.I.D. Software Development, One Step at a Time", Code Magazine, http://www.codemag.com/article/1001061
- [4] Mark Richards, Neal Ford, Fundamentals of Software Architecture: An Engineering Approach, O'Reilly Media; 1st edition (January 28, 2020), ISBN: 978-1492043454
- [5] Len Bass, Paul Clements, Rick Kazman, Software Architecture in Practice, Addison-Wesley Professional; 4th edition (August 3, 2021), ISBN 978-0136886099

Evaluation

- Group-based seminar: 10%
- Group-based project: 40%
- Final exam: 50%

Contact

- Lecturers:
  - Thai-Minh Truong (Trương Thị Thái Minh) (thaiminh@hcmut.edu.vn)

- Course website:
  - https://lms.hcmut.edu.vn

Reference sources of the slides

- Slides in this course are adapted mainly from Richards et al. 2020 [4] and Martin 2017 [1].

[1] Robert C. Martin, Clean Architecture: A Craftsman's Guide to Software Structure and Design, Pearson; 1st edition (September 10, 2017), ISBN 978-0134494166
[4] Mark Richards, Neal Ford, Fundamentals of Software Architecture: An Engineering Approach, O'Reilly Media; 1st edition (January 28, 2020), ISBN: 978-1492043454

CHAPTER 1: INTRODUCTION

Chapter 1. Introduction

1.1. Overview of Design
1.2. Defining Software Architecture
1.3. Expectations of an Architect
1.4. Separation of Concerns
1.5. The Intersection of Software Architecture with DevOps, Processes, and Data

1.1. Overview of Design

"The goal of software architecture is to minimize the human resources required to build and maintain the required system"

"The measure of design quality is simply the measure of the effort required to meet the needs of the customer."

=>*"If that effort is low, and stays low throughout the lifetime of the system, the design is good."*

1.1. Overview of Design

"In most successful software projects, the expert developers working on that project have a shared understanding of the system design. This shared understanding is called architecture."

— Martin Fowler

"Architecture is the ***fundamental organization*** of a system, embodied in its ***components***, their ***relationships*** to ***each other*** and the ***environment***, and the principles governing its ***design and evolution***."

ANSI/IEEE Std 1471-2000

"The software architecture of a program or computing system is the ***structure or structures*** of the system, which comprise ***software elements***, the ***externally visible properties*** of those elements, and the ***relationships*** among them."

SEI- Software Engineering Institute

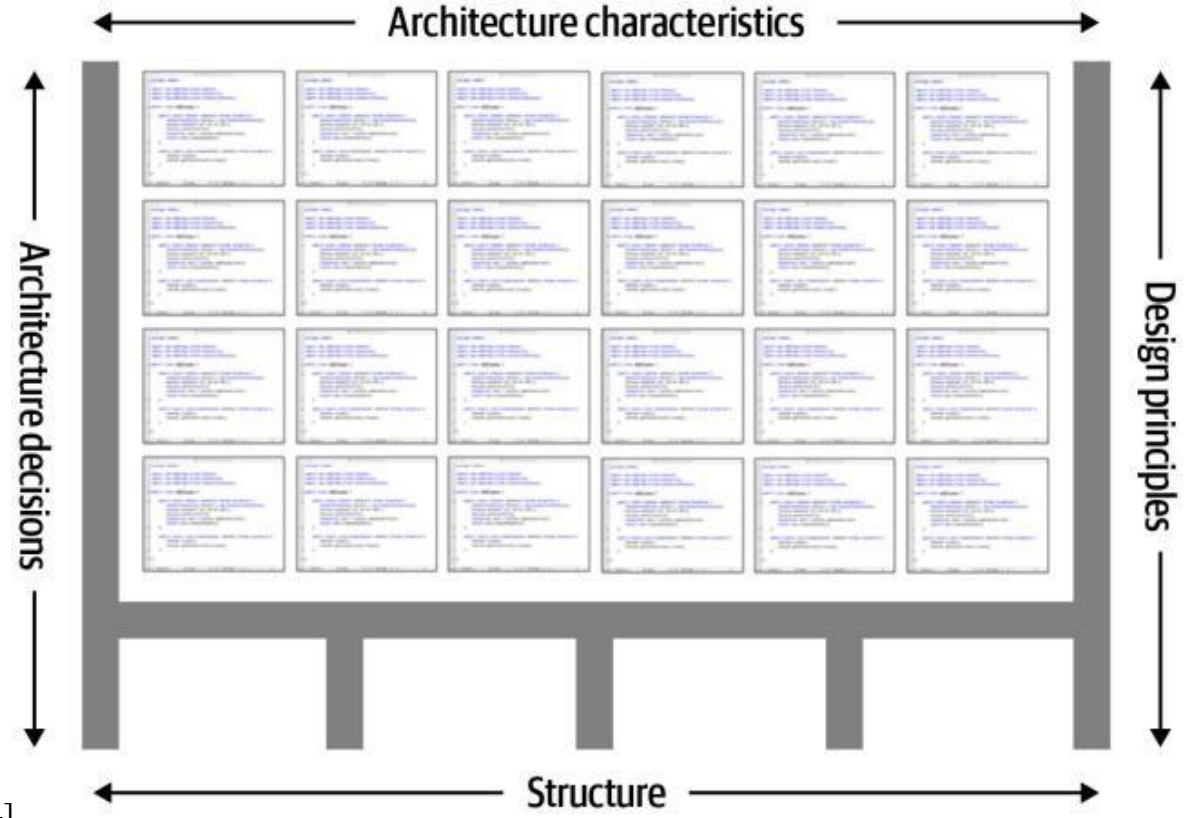
"[Software architecture goes] beyond the algorithms and data structures of the computation; designing and specifying the ***overall system structure*** emerges as a new kind of problem. ***Structural issues*** include gross organization and global control structure; ***protocols for communication, synchronization, and data access; assignment of functionality to design elements; physical distribution; composition of design elements; scaling and performance;*** and ***selection among design alternatives***."

Garlan and Shaw

1.2. Defining Software Architecture

Four dimensions that define software architecture

- Software architecture consists of the structure of the system, combined with architecture characteristics (“-ilities”) the system must support, architecture decisions, and finally design principles.

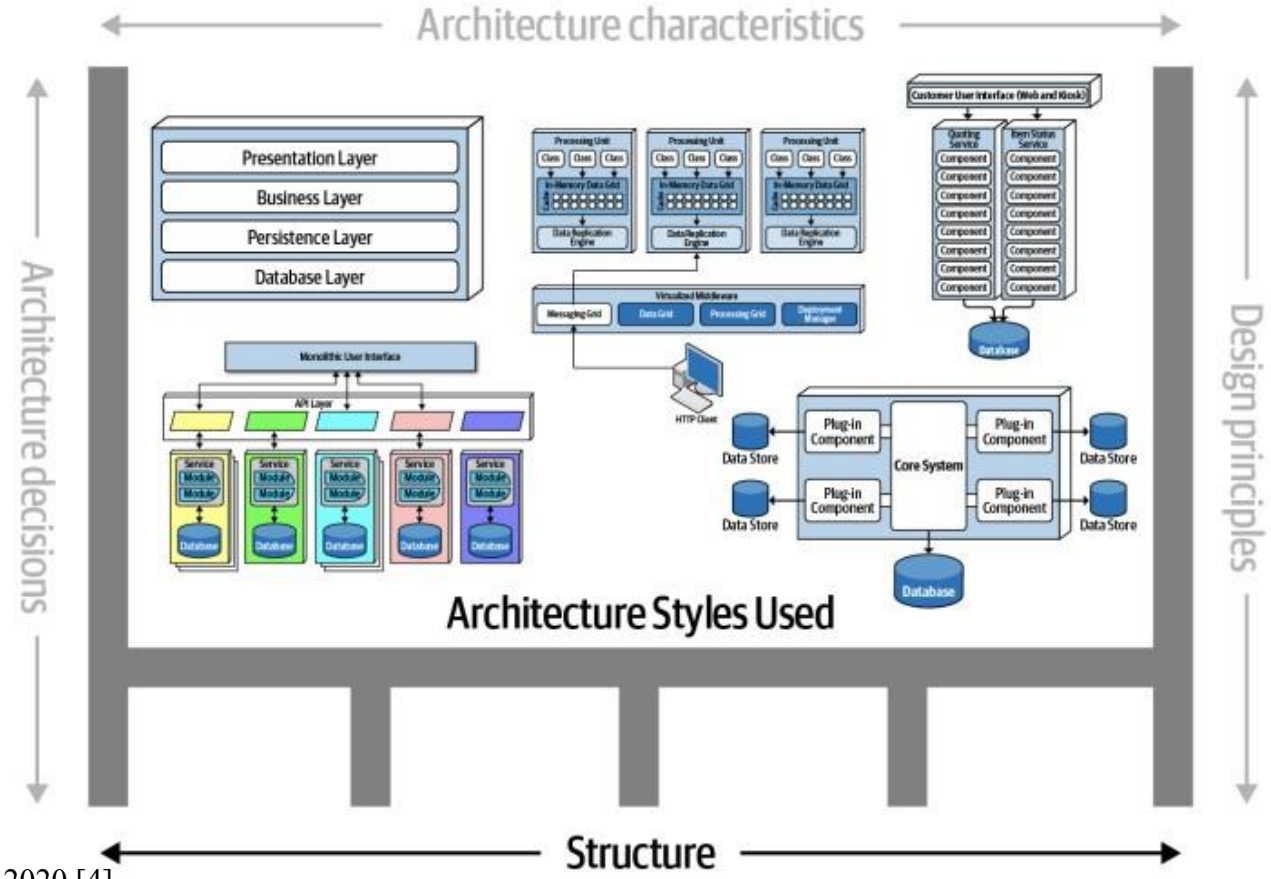


Richards et al. 2020 [4]

1.2. Defining Software Architecture

Structure of the system

- Structure refers to the type of architecture styles used in the system such as microservices, layered, or microkernel

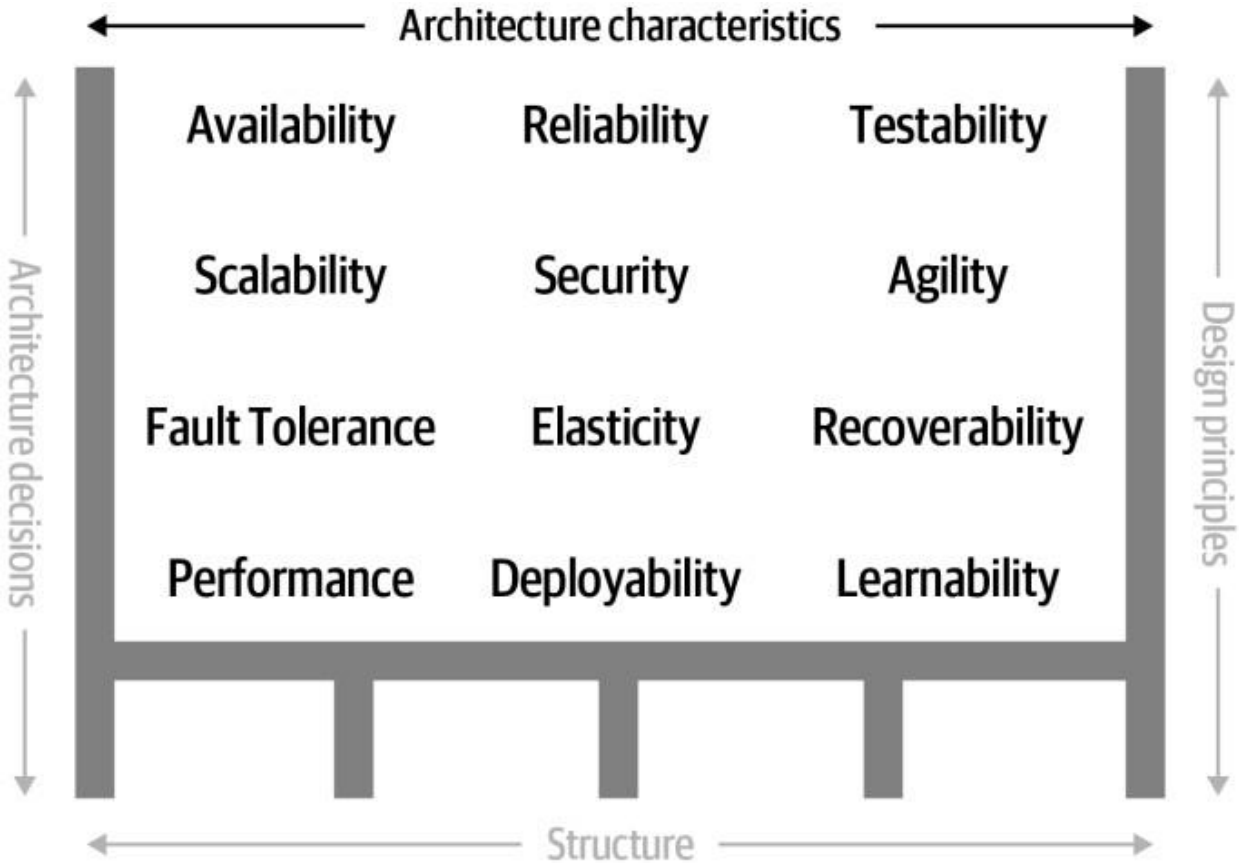


Richards et al. 2020 [4]

1.2. Defining Software Architecture

Architecture characteristics

- The architecture characteristics define the success criteria of a system, which is generally orthogonal to the functionality of the system.

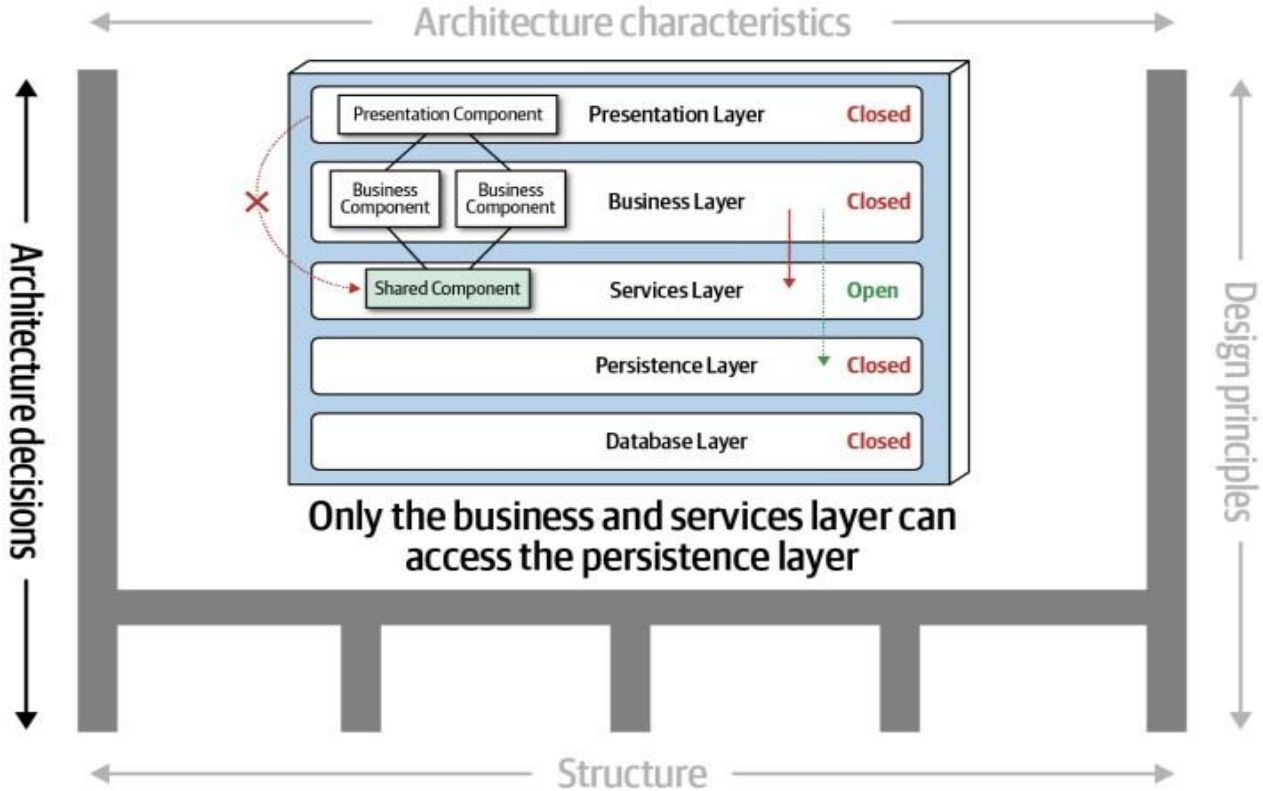


Richards et al. 2020 [4]

1.2. Defining Software Architecture

Architecture decisions

- Architecture decisions define the rules for how a system should be constructed. Architecture decisions form the constraints of the system and direct the development teams on what is and what isn't allowed.

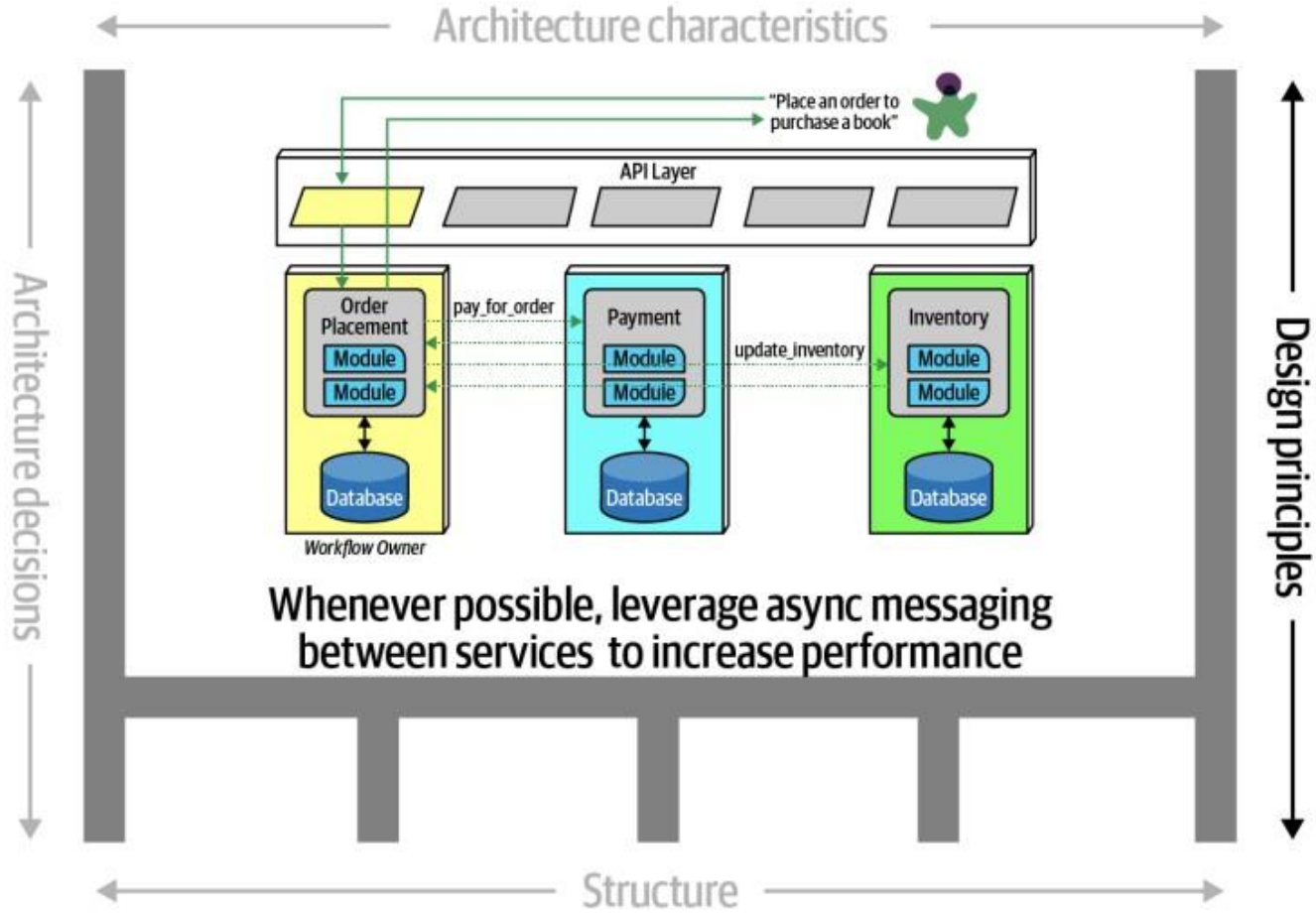


Richards et al. 2020 [4]

1.2. Defining Software Architecture

Design principles

- Guidelines for constructing systems



Richards et al. 2020 [4]

1.3. Expectations of an Architect

Core expectations of a software architect

.....

1.3. Expectations of an Architect

Core expectations of a software architect

1. Make architecture decisions
2. Continually analyze the architecture
3. Keep current with latest trends
4. Ensure compliance with decisions
5. Diverse exposure and experience
6. Have business domain knowledge
7. Possess interpersonal skills
8. Understand and navigate politics.

SA - Laws of Software Architecture

Laws of Software Architecture

First Law

- Everything in software architecture is a trade-off.
  - **Corollary 1:** If an architect thinks they have discovered something that isn't a trade-off, more likely they just haven't identified the trade-off yet.

Second Law

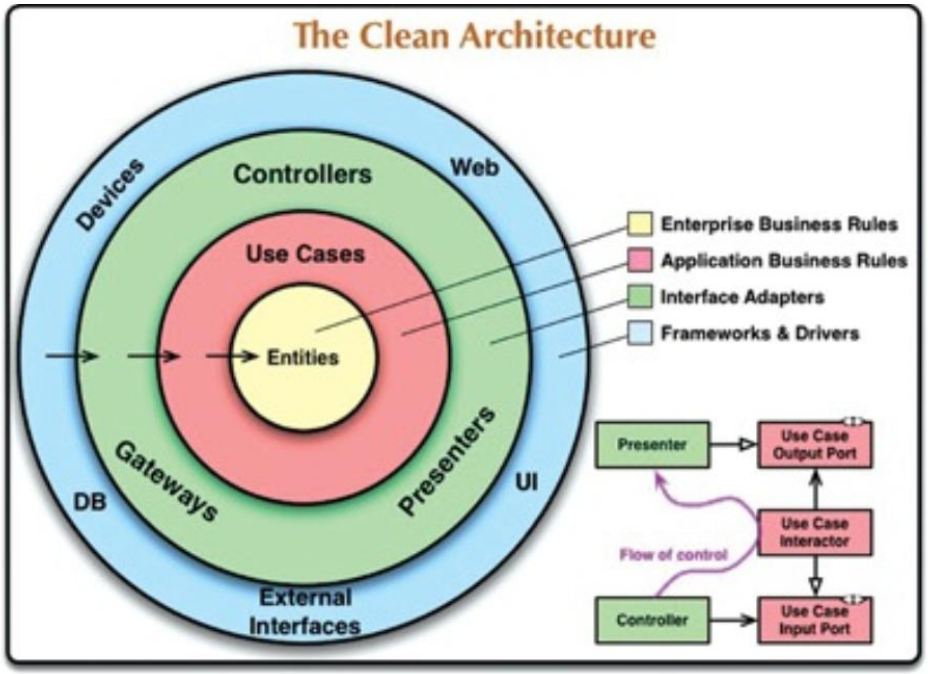
- WHY is more important than HOW.

1.4. Separation of Concerns

- Although software architectures all vary some what in their details, they are very similar.
- They all have the same objective, which is the separation of concerns.
- They all achieve this separation by dividing the software into layers. Each has at least one layer for business rules, and another layer for user and system interfaces.

1.4. Separation of Concerns

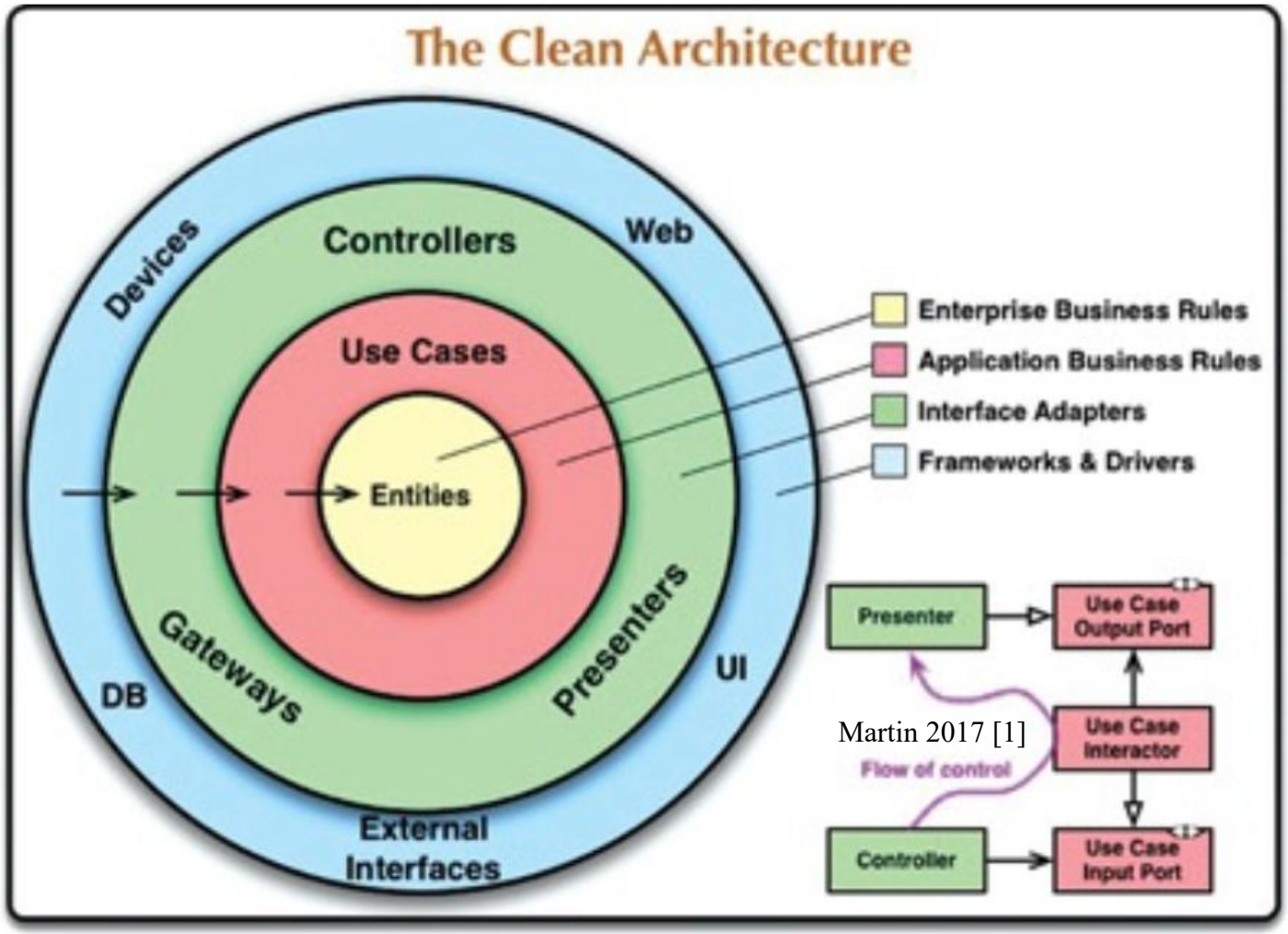
- Each of these architectures produces systems that have the following characteristics:
  - *Independent of frameworks*
  - *Testable.*
  - *Independent of the UI*
  - *Independent of the database.*
  - *Independent of any external agency.*



Martin 2017 [1]

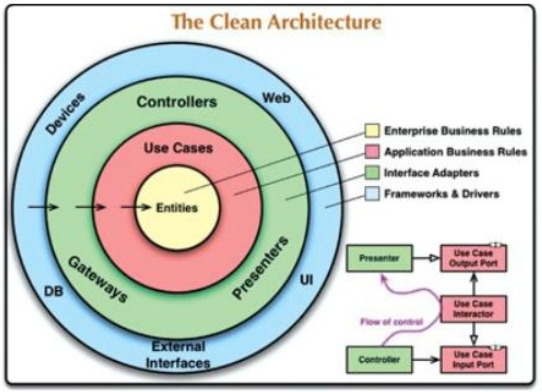
1.4. Separation of Concerns

- The diagram is an attempt at integrating all these architectures into a single actionable idea.



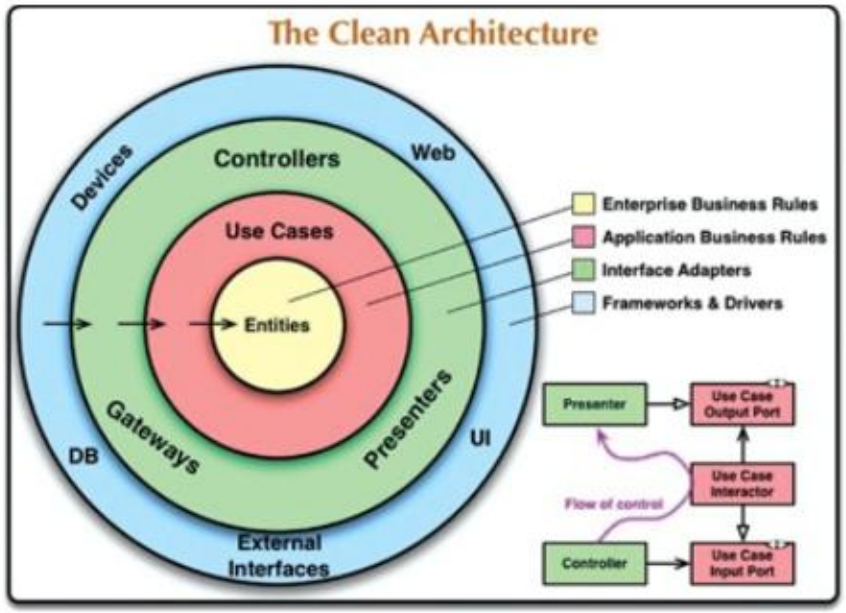
1.4. Separation of Concerns

- **THE DEPENDENCY RULE**: Source code dependencies must point only inward, toward higher-level policies.
- The concentric circles represent different areas of software. In general, the further in you go, the higher level the software becomes. The outer circles are mechanisms. The inner circles are policies.
- Nothing in an inner circle can know anything at all about something in an outer circle. the name of something declared in an outer circle must not be mentioned by the code in an inner circle.
- Data formats declared in an outer circle should not be used by an inner circle.
- We don’t want anything in an outer circle to impact the inner circles.



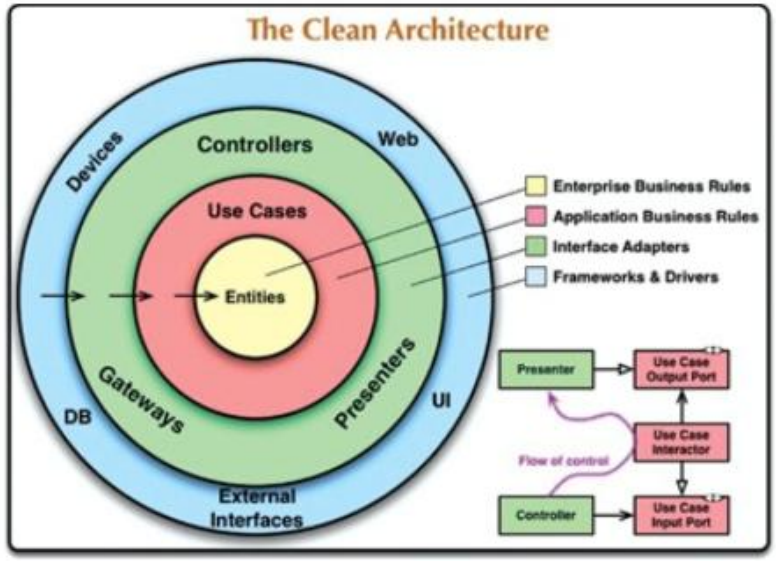
1.4. Separation of Concerns

- **ONLY FOUR CIRCLES?**
- There's no rule that says you must always have just these four.
- However, the Dependency Rule always applies.
- Source code dependencies always point inward.
- As you move inward, the level of abstraction and policy increases.
- The outermost circle consists of low-level concrete details.
- The innermost circle is the most general and highest level.



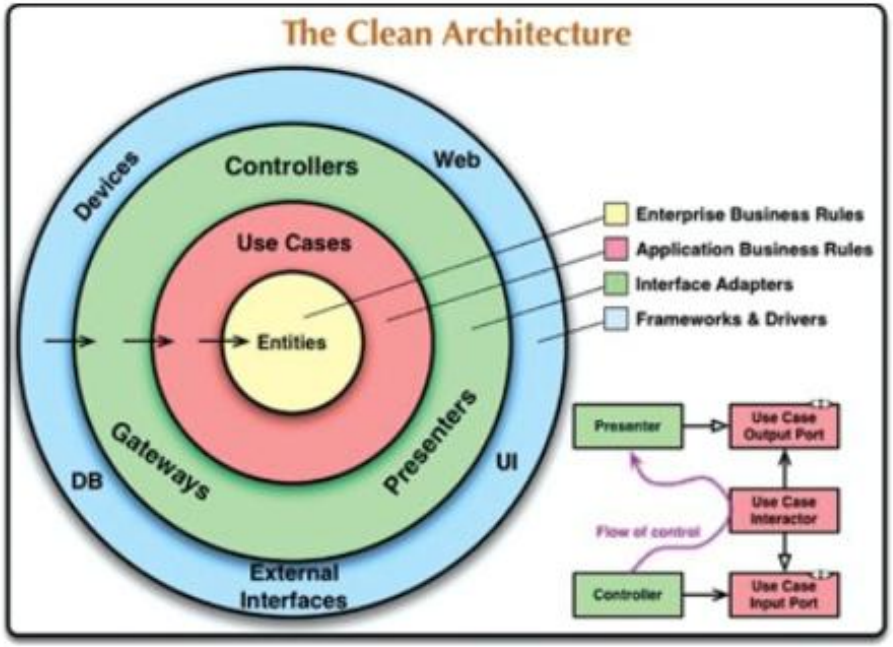
1.4. Separation of Concerns

- **CROSSING BOUNDARIES:**
- The flow of control: It begins in the controller, moves through the use case, and then winds up executing in the presenter.
- The source code dependencies: Each points inward toward the use cases.
- We usually resolve this apparent contradiction by using the Dependency Inversion Principle.
- Arrange interfaces and inheritance relationships such that the source code dependencies oppose the flow of control at just the right points across the boundary.



1.4. Separation of Concerns

- **WHICH DATA CROSSES THE BOUNDARIES:**
- Typically, the data that crosses the boundaries consists of simple data structures, such as: basic structs, simple data transfer objects, arguments in function calls, etc.
- The important thing is that isolated, simple data structures are passed across the boundaries.
- When passing data across a boundary, it is always in the form that is most convenient for the inner circle.



1.5. The Intersection of Software Architecture with DevOps, Processes, and Data

Operations/DevOps

- The most obvious recent intersection between architecture and related fields occurred with the advent of DevOps,
- Many architectures designed during the 1990s and 2000s assumed that architects couldn't control operations
- New forms of architecture that combine many operational concerns with the architecture
  - ESB-driven SOA, Microservices

Process

- Software architecture is mostly orthogonal to the software development process
- The way that you build software (process) has little impact on the software architecture (structure)
- However, the process by which teams develop software has an impact on many facets of software architecture
  - Architects in Agile projects

Data

- A large percentage of serious application development includes external data storage, often in the form of a relational (or, increasingly, NoSQL) database.
- Database administrators often work alongside architects to build data architecture for complex systems, analyzing how relationships and reuse will affect a portfolio of applications.

Q&A